



Cigna-SPC

Training & Learning Management

ETL - Processing - End-to-End Data Engineering Project

Ikjot Kour - 14184237

Accenture AEH Intern | Custom Associate Software Engineer



PROJECT DESCRIPTION

What is ETL – LTM?

An end-to-end ETL pipeline that tracks employee learning and training activities.

Project Overview

The LTM (Learning & Training Management) pipeline ingests raw training data from CSV, Excel and JSON files, applies multi-layer transformations, and persists a clean, auditable history in a MySQL data warehouse.

Built during the Accenture internship using Python and MySQL, the pipeline follows the industry-standard Three Layer Pipeline Architecture — Landing, Staging and Base layers.

SCD Type 2 logic ensures full historical tracking of every data change — no record is ever lost or overwritten.

5

Source Tables

3

Pipeline Layers

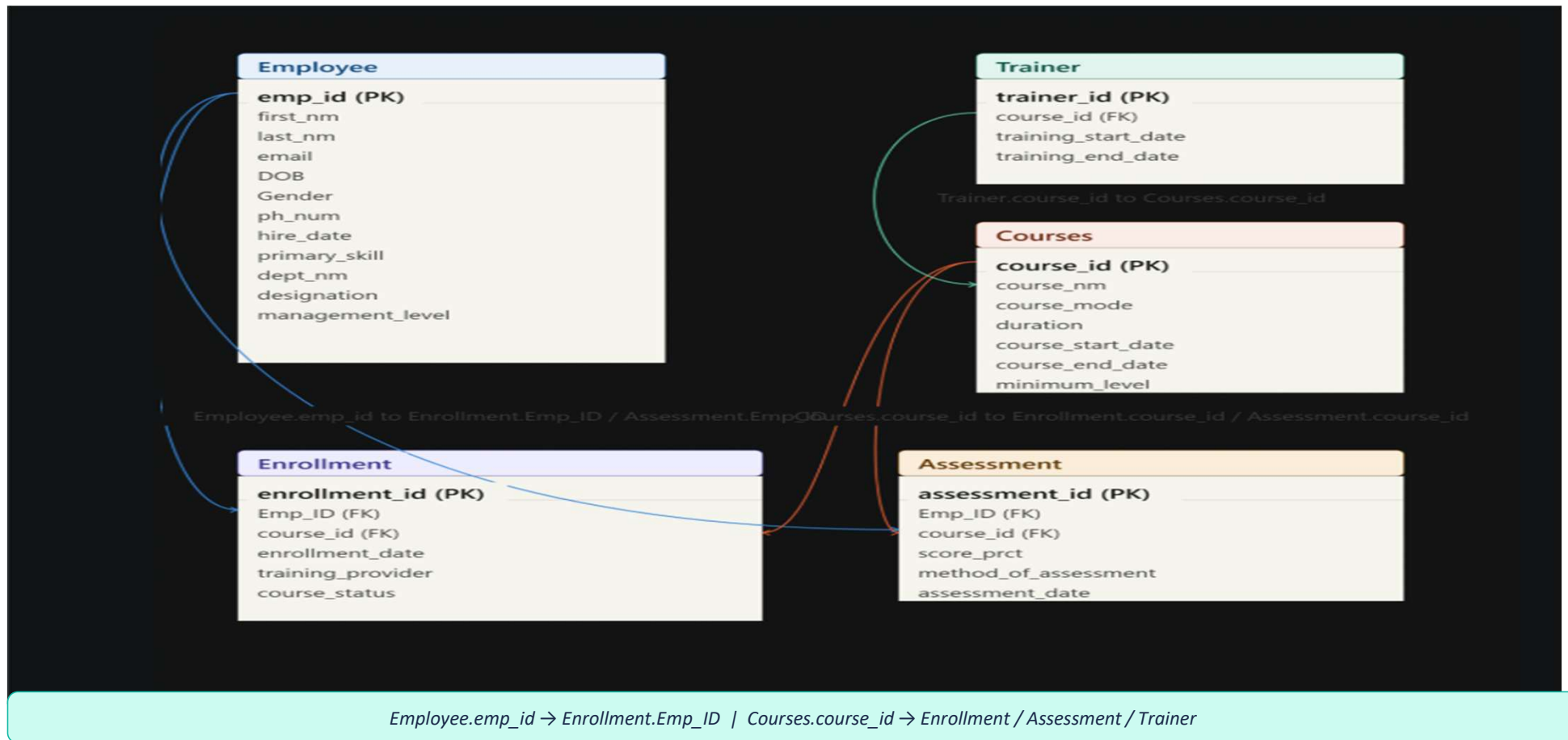
SCD2

History Method

MD5

Checksum Type

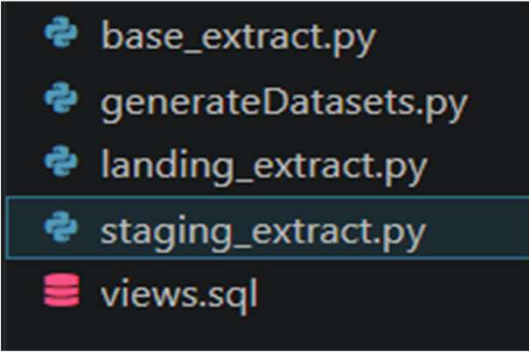
Entity Relationship Diagram



Project Files & Dataset Generation

Dataset generation was a core part of the project — realistic data built to reflect real-world training scenarios.

Project Structure



```
base_extract.py
generateDatasets.py
landing_extract.py
staging_extract.py
views.sql
```

landing_extract.py

Ingests raw files, applies ID padding, writes to MySQL landing tables with load_key and audit trail

staging_extract.py

Transforms data, computes MD5 checksum, assigns change_flag (I/U/N), inserts into staging tables

base_extract.py

Reads change_flag from staging, applies SCD Type 2 logic, populates base layer, clears staging

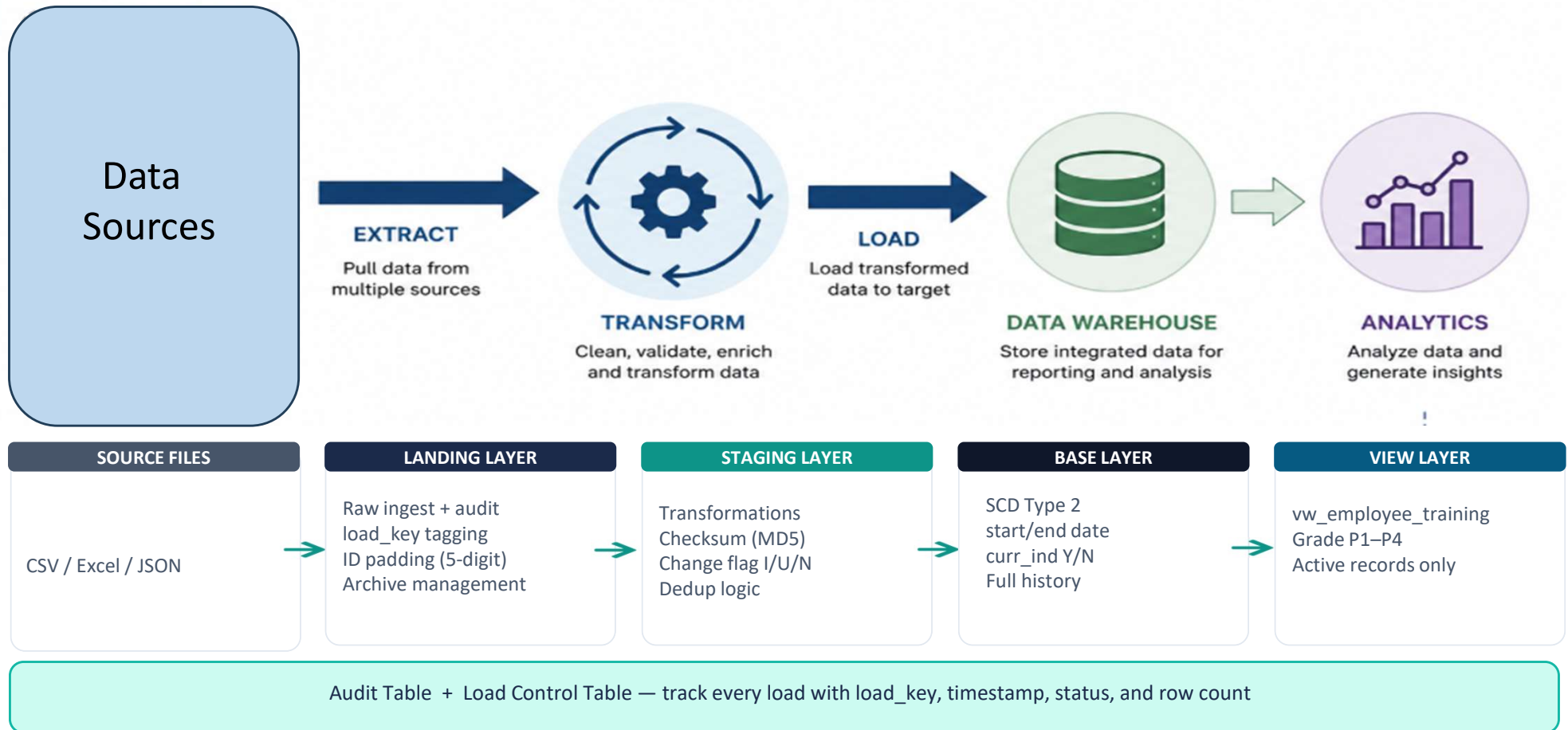
generateDatasets.py

Generates realistic CSV datasets with skill-to-course mapping, management level eligibility and multi-day load simulation

views.sql

Creates vw_employee_training joining employee, enrollment and assessment with P1–P4 grade bands

Three-Layer Pipeline Architecture



Load Control & Audit Framework

Two metadata tables give complete traceability across every stage — Landing, Staging and Base.

Load Control Table

One row per ETL execution. Generates the run's Load Key, shared across Landing, Staging and Base.

control_id	project_name	load_key	load_date	status
1	LTM	LCK_20260707_231149_XAJIOY	2026-07-07 23:11:50	success
2	LTM	LCK_20260707_231234_XAJIOY	2026-07-07 23:12:35	success
3	LTM	LCK_20260707_231304_XAJIOY	2026-07-07 23:13:05	success
4	LTM	LCK_20260707_231334_XAJIOY	2026-07-07 23:13:35	success

Audit Table

One row per processing step, for every stage — Landing, Staging and Base alike.

audit_id	table_name	file_name	ingest_timestamp	load_key	status	file_path	row_count
15	base_assessment	base_extract.py	2026-07-07 23:12:17	LCK_20260707_231149_XAJIOY	success	source	282
16	landing_employee	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source	3
17	landing_course	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source	90
18	landing_trainer	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source	9
19	landing_enrollment	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source	784
20	landing_assessment	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source	276
21	staging_employee	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	source	3
22	staging_course	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	source	7
23	staging_trainer	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	source	7
24	staging_enrollment	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	source	784
25	staging_assessment	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	source	273
26	base_employee	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	source	3
27	base_course	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	source	50
28	base_trainer	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	source	7
29	base_enrollment	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	source	784
30	base_assessment	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	source	273
31	landing_employee	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source	0
32	landing_course	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	no_file_found	source	0
33	landing_trainer	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source	0
34	landing_enrollment	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source	0
35	landing_assessment	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source	0
36	landing_employee	landing_extract.py	2026-07-07 23:13:35	LCK_20260707_231334_XAJIOY	success	source	3
37	landing_course	landing_extract.py	2026-07-07 23:13:35	LCK_20260707_231334_XAJIOY	no_file_found	source	0

What Each Audit Column Means

- table_name** The target table the record was processed into, e.g. landing_employee, staging_course, base_trainer.
- load_key** The run identifier from Load Control, shared across all three layers for that execution.
- file_path** The raw source file path — populated only for Landing entries; NULL for Staging and Base, since those stages read from tables, not files.

- file_name** The ETL script that performed the step — landing_extract.py, staging_extract.py or base_extract.py. Populated for every stage.
- ingest_timestamp** The date/time that processing step ran.
- row_count** Number of rows processed by that step.

Landing Layer

Raw ingestion with full audit trail — no transformation, only ingestion tracking.

- Reads CSV / Excel / JSON from source folder using pandas
- Picks lowest-date file first per table (oldest unprocessed load first)
- Pads IDs to 5 digits after prefix: E0001, C00023, T00008, En00105, As00441
- Normalizes column names to lowercase_underscore format
- Inserts rows via raw SQL cursor.executemany() — no SQLAlchemy / df.to_sql()
- Adds ingest_timestamp and load_key to every row
- Records one audit_table row per file
- Writes to load_control: project=LTM, load_key, status (STARTED → success/partial)
- Moves processed files to archive folder, keeps last 7 per prefix

ingest_timestamp	load_key
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:11:49.536755	LCK_20260707_231149_XAJIOY
2026-07-07 23:12:34.816646	LCK_20260707_231234_XAJIOY
2026-07-07 23:12:34.816646	LCK_20260707_231234_XAJIOY
2026-07-07 23:12:34.816646	LCK_20260707_231234_XAJIOY
2026-07-07 23:12:34.816646	LCK_20260707_231234_XAJIOY

Staging Layer

Handles exactly one load at a time. Applies all transformations and change detection.

Transformations Applied

- Column casing standardized; whitespace stripped
- Name fields title-cased
- Dates converted to YYYY-MM-DD
- Email validated via regex; phone limited to 7–15 digits
- Gender standardized to M / F only (e.g. Male → M)
- Assessment score constrained: $0 < \text{score} < 100$
- Start date > end date → NULL (course, trainer)
- Null / blank / NaN values → SQL NULL

Checksum & Change Flag

Only records flagged I or U enter Staging.

- Checksum: MD5 hash of the original archive data
- Excludes ingest_timestamp and load_key from the hash
- Sorted column order for a consistent 32-char hex output

I

New ID — never seen in Base

U

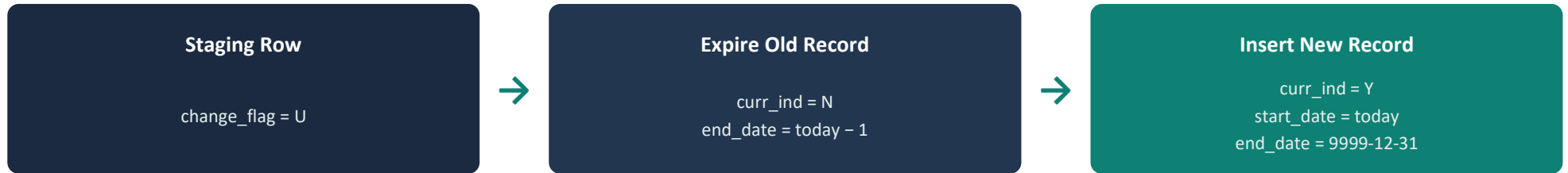
ID exists — checksum differs

N

No change — row dropped

Base Layer — SCD Type 2

Full history preserved. Every change creates a new version; the old record is expired.



Example — Employee E00001 changes department:

employee_key	emp_id	dept_nm	start_date	end_date	curr_ind
E00001_20240101120000	E00001	IT Operations	2024-01-01	2026-06-25	N
E00001_20260626093000	E00001	Software Development	2026-06-26	9999-12-31	Y

At most one row per business key may have curr_ind = 'Y' at any time.

Why Does This Pipeline Exist?

Four outcomes the LTM pipeline delivers for the business.



Centralized Training Visibility

Consolidates employee, course, enrollment and assessment data from siloed source files into a single queryable warehouse.



Auditable Change History

SCD Type 2 ensures every change to an employee or course record is preserved — who changed, what changed, and when.



Automated Data Quality

Landing-to-staging transformations enforce email validity, phone format, gender standardization, date integrity and null handling.



Actionable Reporting via Views

vw_employee_training surfaces grade bands (P1–P4) and course status, ready for BI consumption without re-querying raw tables.

SAMPLE DATA

vw_employee_training — View Output

Joins Employee → Enrollment → Assessment. Active records only. Grade derived from score_prct.

emp_id	first_nm	last_nm	email	primary_skill	management_level	course_id	course_nm	course_status	score_prct	grade	assessment_status
E00011	Maria	Gordon	ellisonkendra@example.org	Campaign_Management	11	C00029	Campaign Management Essentials	Enrolled	65	P3	Passed
E00012	Robert	Brown	dilloncruz@example.org	Bug_Tracking	9	C00014	Ethical Hacking and Penetration Testing	Cancelled	NULL	NULL	Cancelled
E00012	Robert	Brown	dilloncruz@example.org	Bug_Tracking	9	C00025	Product Roadmapping and Strategy	Enrolled	82	P4	Passed
E00013	Kenneth	Hall	amandamoore@example.net	Escalation_Management	11	C00050	Escalation Management for Support Teams	Completed	84	P4	Passed
E00014	James	Bolton	connerjennifer@example.org	HRIS_Systems	7	C00025	Product Roadmapping and Strategy	Cancelled	NULL	NULL	Cancelled
E00015	Kathryn	Sanchez	greenjoseph@example.net	Version Control - Git	7	C00025	Product Roadmapping and Strategy	Enrolled	56	P2	Retest
E00016	Brandon	Walker	ghoffman@example.com	Visualization - Tableau	8	C00025	Product Roadmapping and Strategy	Cancelled	NULL	NULL	Cancelled
E00017	Emily	Coleman	evanharris@example.org	Agile - Scrum	10	C00021	Agile Scrum Master Training	Completed	55	P2	Retest
E00017	Emily	Coleman	evanharris@example.org	Agile - Scrum	10	C00014	Ethical Hacking and Penetration Testing	Completed	52	P2	Retest
E00017	Emily	Coleman	evanharris@example.org	Agile - Scrum	10	C00033	Payroll Management Certification	Cancelled	NULL	NULL	Cancelled
E00018	Matthew	Davis	johnbecker@example.org	Campaign_Management	11	C00029	Campaign Management Essentials	Enrolled	63	P3	Passed
E00018	Matthew	Davis	johnbecker@example.org	Campaign_Management	11	C00039	Interviewing Skills for Recruiters	Completed	96	P4	Passed
E00018	Matthew	Davis	johnbecker@example.org	Campaign_Management	11	C00008	Tableau for Data Visualization	In Progress	94	P4	Passed
E00019	Ann	Orozco	taustin@example.net	Recruitment - Sourcing	11	C00038	Recruitment and Talent Sourcing	Enrolled	70	P3	Passed
E00019	Ann	Orozco	taustin@example.net	Recruitment - Sourcing	11	C00020	Azure Cloud Fundamentals	Cancelled	NULL	NULL	Cancelled

● P1 0 – 40 ● P2 41 – 60 ● P3 61 – 80 ● P4 81 – 100

This single view gives BI tools a ready-to-use reporting dataset — no need to re-query raw Employee, Enrollment or Assessment tables.

P1 , P2 , P3 , P4 = Grades based on score_prct

TESTING

What was Tested ?

A → B Testing

Left Anti Join confirms every key in the source layer exists in the target — a non-zero result flags missing records.

Landing → Staging: 0 missing records ✓

Staging → Base: 0 missing records ✓

Landing → Base: 0 missing records ✓

No records were lost during ETL processing.

Mandatory Field Testing

Ensures records with missing mandatory business keys are rejected before entering the Staging layer.

- NULL assessment_id : 2
- Action If found: **Records Rejected**

Only records with valid mandatory IDs were loaded into Staging

Duplicate Key Testing

Checks emp_id, course_id, trainer_id, enrollment_id, and assessment_id for duplicate values within a single load batch.

Duplicate found assessment_id : A00003

Action if found: **Duplicate Skipped**

Only one valid record was inserted into Staging.

The ETL pipeline successfully detected missing mandatory fields and duplicate business keys, ensuring only valid and unique records progressed to the Staging layer.

assessment_id	Emp_ID	course_id	score_prct	method_of_assessment
nan	E00013	C00050	71	Hands-on
nan	E00017	C00033	84	Applied Project Simulation
A00003	E00018	C00029	41	Hands-on
A00003	E00018	C00029	41	Hands-on
A00004	E00018	C00009	60	Hands-on



assessment_id	emp_id	course_id	score_prct	method_of_assessment	assessment_date
A00003	E00018	C00029	41	Hands-on	2026-09-02
A00004	E00018	C00009	60	Hands-on	2021-12-23
A00005	E00019	C00025	60	Hands-on	2023-01-24
A00006	E00021	C00027	99	Applied Project Simulation	2024-10-27
A00007	E00023	C00025	57	Hands-on	2023-09-11

Scaling to Billions of Rows

Optimizations applied in Python + MySQL, alongside reporting insights from the Base layer.



Chunked Processing

LIMIT/OFFSET reads fixed-size batches instead of loading the full landing table into memory at once — bounded RAM regardless of table size.



Dict-based Archive Lookup

Archive file indexed as {id: row} once per table ($O(1)$ lookup), replacing a linear scan per row — from $O(n \times m)$ to $O(n)$.

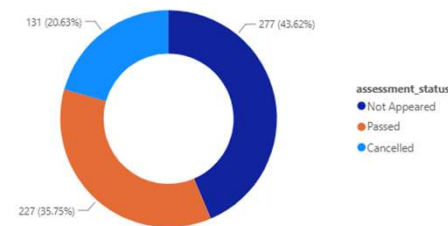


Landing → Bronze (Auto Loader)

Watches cloud storage and auto-ingests new files incrementally, replacing the find_lowest_date_file logic.

Assessment Status

Count of emp_id by assessment_status



277 Not Appeared

227 Passed

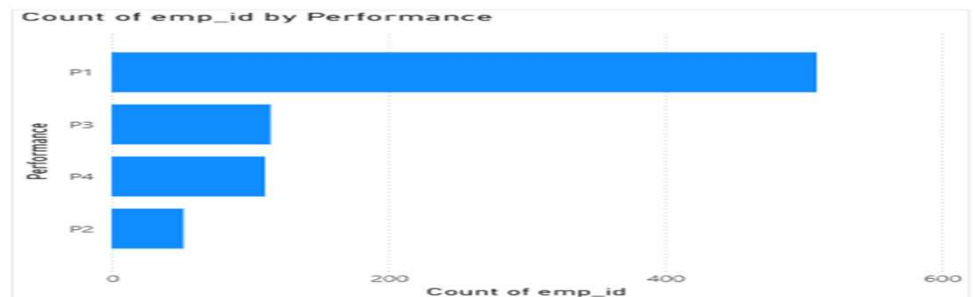
131 Cancelled

227
Passed

277
Not Appea...

131
Cancelled

Employees by Performance Band



Thank You

LTM — Learning & Training Management ETL Pipeline

Ikjot Kour

Data Engineering Intern, Accenture

Python · MySQL · ETL · SCD Type 2 · Three Layer Pipeline Architecture